

ল্যাংচেইনে Prompts: সহজ ও বাস্তব উদাহরণসহ বিস্তারিত ব্যাখ্যা

অনেক নতুন ডেভেলপার যখন LangChain শেখা শুরু করে, তখন তারা মনে করে Prompt মানে শুধু ChatGPT-কে কিছু লিখে পাঠানো। কিন্তু বাস্তবে Prompt হলো পুরো AI Application-এর সবচেয়ে গুরুত্বপূর্ণ অংশগুলোর একটি। আপনি LLM-কে কীভাবে নির্দেশনা দিচ্ছেন, সেটার উপরই অনেকাংশে নির্ভর করে সে কতটা ভালো উত্তর দেবে।

একই Model ব্যবহার করে দুইজন Developer সম্পূর্ণ ভিন্ন মানের Application বানাতে পারে। কারণ একজন Prompt Design ভালো জানে, আর অন্যজন জানে না।

Prompt আসলে কী?

সহজ ভাষায়, LLM-কে আমরা যে নির্দেশনা বা Message পাঠাই, সেটাই Prompt।

ধরুন আপনি ChatGPT-কে লিখলেন:

Write a short paragraph about Bangladesh.

এটাই Prompt।

আবার যদি লিখেন:

Explain Machine Learning in simple terms.

এটাও Prompt।

LLM মানুষের মনের কথা বুঝতে পারে না। সে শুধু Prompt দেখে বোঝার চেষ্টা করে আপনি আসলে কী চান।

তাই Prompt যত পরিষ্কার হবে, উত্তরও তত ভালো হবে।

ধরুন আপনি একজন কর্মচারীকে বললেন:

Create a report.

এখানে সে বুঝতে পারবে না কী রিপোর্ট বানাতে হবে।

কিন্তু যদি বলেন:

Create a 300-word report about the impact of AI on education.

তাহলে কাজটা অনেক পরিষ্কার হয়ে যায়।

LLM-এর ক্ষেত্রেও একই বিষয় প্রযোজ্য।

Temperature কী এবং কেন গুরুত্বপূর্ণ?

Temperature হলো এমন একটি Setting যা Model কতটা Creative হবে তা নিয়ন্ত্রণ করে।

সহজভাবে মনে রাখুন:

কম **Temperature** = বেশি স্থির উত্তর

বেশি **Temperature** = বেশি সৃজনশীল উত্তর

Temperature = 0

ধরুন Prompt:

What is the capital of France?

Temperature যদি 0 হয়, তাহলে প্রায় প্রতিবার উত্তর আসবে:

Paris

কারণ Model সবচেয়ে সম্ভাব্য উত্তরটাই বেছে নিচ্ছে।

একই প্রশ্ন ১০ বার করলেও প্রায় একই উত্তর পাবেন।

এটাকে Deterministic Output বলা হয়।

এ ধরনের Setting সাধারণত ব্যবহার করা হয়:

- Question Answering
- Data Analysis
- Information Extraction
- Summarization

কারণ এখানে Creativity দরকার নেই, Accuracy দরকার।

Temperature = 1.5 বা 2

এবার Prompt:

Write a short story about a dragon.

প্রথমবার Model লিখতে পারে:

A dragon lived on a mountain and protected an ancient treasure.

দ্বিতীয়বার:

A young dragon dreamed of becoming the king of the skies.

তৃতীয়বার:

A lonely dragon traveled across the world searching for friends.

দেখুন, Prompt একই কিন্তু উত্তর ভিন্ন।

কারণ Temperature বেশি থাকলে Model নতুন নতুন সম্ভাবনা থেকে শব্দ বেছে নেয়।

এটি ব্যবহার করা হয়:

- Story Writing
- Content Creation
- Marketing Copy
- Poetry
- Brainstorming

Static Prompt কী?

Static Prompt হলো এমন Prompt যা আগে থেকেই কোডে লিখে রাখা হয়।

উদাহরণ:

prompt = "Write a summary about Artificial Intelligence."

এখানে Prompt সবসময় একই থাকবে।

User যাই ইনপুট দিক, Prompt পরিবর্তন হবে না।

ধরুন আপনার Website-এ ১০০ জন User এসেছে।

কেউ Machine Learning-এর Summary চায়।

কেউ Deep Learning-এর Summary চায়।

কেউ NLP-এর Summary চায়।

Static Prompt দিয়ে এটা Handle করা কঠিন।

কারণ Prompt Fixed।

Dynamic Prompt কী?

Dynamic Prompt-এর ক্ষেত্রে Prompt-এর কিছু অংশ User-এর Input অনুযায়ী পরিবর্তিত হয়।

ধরুন Template:

template = "Write a summary about {topic}"

যদি User দেয়:

Machine Learning

তাহলে Prompt হবে:

Write a summary about Machine Learning

আবার যদি User দেয়:

Deep Learning

তাহলে Prompt হবে:

Write a summary about Deep Learning

একই Template ব্যবহার করে হাজারো Prompt তৈরি করা যায়।

এ কারণেই বাস্তব Application-এ Dynamic Prompt বেশি ব্যবহার করা হয়।

PromptTemplate কেন ব্যবহার করা হয়?

এখন প্রশ্ন হলো, Python-এর f-string দিয়েই তো এটা করা যায়।

যেমন:

```
topic = "Machine Learning"
```

```
prompt = f"Write a summary about {topic}"
```

তাহলে PromptTemplate আবার কেন?

কারণ বড় Project-এ PromptTemplate অনেক অতিরিক্ত সুবিধা দেয়।

Validation

ধরুন Template-এ দুইটি Variable আছে:

```
template = """  
Write a summary about {topic}  
in {language}  
"""
```

কিন্তু আপনি শুধু Topic দিলেন:

```
topic = "Machine Learning"
```

Language দিতে ভুলে গেলেন।

PromptTemplate সঙ্গে সঙ্গে Error দেখাবে।

ফলে Bug সহজে ধরা যায়।

Reusability

একবার Template লিখলে সেটি বারবার ব্যবহার করা যায়।

ধরুন আপনি Blog Generator বানাচ্ছেন।

একই Template দিয়ে হাজারো Blog তৈরি করা সম্ভব।

LangChain Integration

PromptTemplate সরাসরি LangChain-এর অন্যান্য Component-এর সাথে কাজ করে।

যেমন:

- Chains

- Agents
- Memory
- RAG

এ কারণে বড় AI Project-এ PromptTemplate ব্যবহার করাই Standard Practice।

Chatbot-এর Memory সমস্যা

এখানেই বেশিরভাগ নতুন Developer কনফিউজ হয়।

তারা ভাবে ChatGPT সবকিছু মনে রাখে।

আসলে LLM-এর নিজস্ব কোনো Memory নেই।

ধরুন Conversation:

User:

My name is Rahim.

AI:

Nice to meet you, Rahim.

কিছুক্ষণ পরে User লিখল:

What is my name?

যদি Model-এর কাছে আগের Message না পাঠানো হয়, তাহলে সে জানবে না আপনার নাম Rahim।

কারণ Model শুধু বর্তমান Input দেখে।

সে আগের Conversation নিজে থেকে মনে রাখতে পারে না।

তাহলে Chatbot Memory কীভাবে কাজ করে?

এখানে একটা ছোট Trick ব্যবহার করা হয়।

প্রতিবার নতুন প্রশ্ন আসলে আগের কথোপকথনও Prompt-এর সাথে পাঠানো হয়।

ধরুন User নতুন করে জিজ্ঞেস করল:

What is my name?

LLM-এর কাছে আসলে যা পাঠানো হয়:

User: My name is Rahim.

Assistant: Nice to meet you, Rahim.

User: What is my name?

এখন Model বুঝতে পারবে:

Your name is Rahim.

এটাই Chat History-এর মূল ধারণা।

LangChain-এর তিন ধরনের Message

System Message

System Message AI-এর Personality সেট করে।

উদাহরণ:

You are a professional Python instructor.

এখন AI Programming Teacher-এর মতো উত্তর দেবে।

আরেকটি উদাহরণ:

You are a senior software architect.

এখন AI Architecture-related উত্তর দিতে চেষ্টা করবে।

System Message পুরো Conversation-কে নিয়ন্ত্রণ করে।

Human Message

User যা লিখে, সেটাই Human Message।

উদাহরণ:

Explain decorators in Python.

এটি Human Message।

AI Message

Model যে উত্তর দেয়, সেটাই AI Message।

উদাহরণ:

A decorator is a function that modifies the behavior of another function.

এটি AI Message।

বাস্তব Chatbot-এ এই তিনটি Message একসাথে কাজ করে।

ChatPromptTemplate কী?

যখন শুধু একটি Prompt নয়, বরং অনেকগুলো Message নিয়ে কাজ করতে হয়, তখন ChatPromptTemplate ব্যবহার করা হয়।

ধরুন আপনি এমন একটি Chatbot বানাচ্ছেন যেখানে:

- System Message আছে
- Previous Conversation আছে
- Current User Question আছে

সবকিছু সুন্দরভাবে Manage করার জন্য ChatPromptTemplate ব্যবহার করা হয়।

এটি মূলত Chatbot-এর জন্য PromptTemplate-এর Advanced Version।

MessagesPlaceholder কী?

এখন সবচেয়ে গুরুত্বপূর্ণ অংশ।

ধরুন আপনার Database-এ User-এর আগের ৫০টি Message Save করা আছে।

আপনি চান নতুন Prompt-এর মধ্যে সেই History যোগ করতে।

সেখানেই MessagesPlaceholder ব্যবহার করা হয়।

ধরুন Template-এর Structure এমন:

System Message

Previous Chat History

Current User Question

এখানে "Previous Chat History" অংশটিই MessagesPlaceholder দিয়ে পূরণ করা হয়।

Database থেকে History এনে LangChain সেখানে বসিয়ে দেয়।

ফলে Model আগের Context বুঝতে পারে।

বাস্তব উদাহরণ

ধরুন User-এর Conversation:

User: My name is Rahim.

Assistant: Nice to meet you.

User: I live in Dhaka.

Assistant: That's great.

এখন User লিখল:

Where do I live?

MessagesPlaceholder ব্যবহার করলে LLM-এর কাছে পুরো Conversation যাবে।

ফলে Model উত্তর দেবে:

You live in Dhaka.

কিন্তু History না পাঠালে Model উত্তর দিতে পারবে না।